

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №1

“Основи нереляційних баз даних NoSQL”

Виконав:

Студент 3 курсу

групи Фел-33

Абдулханов Абдул-Рахім

Викладач:

ас. Жишкович А. В.

Львів-2026

Мета роботи: Опанувати основи концепції нереляційних баз даних NoSQL, проаналізувати їхні фундаментальні переваги та обмеження при побудові високонавантажених розподілених систем (HighLoad & Big Data). Дослідити причини переходу від класичної реляційної моделі (RDBMS) до NoSQL, засвоїти теоретичні засади розподілених систем (теорема CAP, порівняння моделей ACID та BASE), вивчити стратегії масштабування (Scale-Up vs Scale-Out, шардинг, реплікація) та навчитися приймати обґрунтовані архітектурні рішення щодо вибору СУБД та застосування багатоваріантної персистентності (Polyglot Persistence).

Теоретичні відомості:

Протягом кількох десятиліть реляційні бази даних (RDBMS: Oracle, PostgreSQL, MySQL) залишалися абсолютним стандартом для збереження структурованих даних. Вони базуються на реляційній алгебрі, фіксованій табличній схемі (Schema-on-Write) та суворій гарантії транзакційної цілісності за моделлю ACID (Atomicity, Consistency, Isolation, Durability).

Однак стрімкий розвиток інтернету, соціальних мереж, мобільних додатків та Інтернету речей (IoT) призвів до появи феномену Великих Даних (Big Data), які описуються властивостями 3V (Volume - терабайти й петабайти, Velocity - мільйони запитів на секунду, Variety - різноманітні слабоструктуровані формати). Реляційні бази зіткнулися з двома критичними бар'єрами: неможливістю дешевого горизонтального масштабування (Scale-Out) через надмірні накладні витрати на виконання багатотабличних операцій JOIN та складністю модифікації жорстких фіксованих схем при динамічній зміні бізнес-вимог.

Як відповідь на ці виклики виник рух NoSQL ('Not Only SQL'). Термін NoSQL уперше запропонував Карло Строщі (Carlo Strozzi) у 1998 р., а в сучасному розумінні як альтернативу реляційним системам його відродив Йохан Оскарссон (Johan Oskarsson) у 2009 р. на зустрічі розробників розподілених сховищ. NoSQL СУБД відмовилися від жорсткої схеми на користь агрегатно-орієнтованих моделей (документи, пари ключ-значення, ширококолонкові структури, графи, а також векторні сховища для штучного інтелекту) і перейшли від суворої узгодженості ACID до більш гнучкої моделі BASE (Basically Available, Soft state, Eventual consistency). У розподілених середовищах вибір характеристик СУБД регламентується фундаментальною теоремою CAP (сформульованою Еріком Брюером у 2000 р. та доведеною Сетом Гілбертом і Ненсі Лінч у 2002 р.), яка стверджує, що при неминучих збоях мережевого зв'язку (Partition) розподілена система змушена обирати між строгою узгодженістю даних (CP) та безперервною доступністю сервісу (AP).

Виконання роботи

1. Порівняння реляційних (SQL) та нереляційних (NoSQL) баз даних:

Для розуміння фундаментальних відмінностей між технологіями склав порівняльну характеристику за 5 ключовими архітектурними параметрами:

- Структура даних:

- * SQL: строго реляційна, таблична модель із фіксованими стовпцями, рядками та зовнішніми ключами (Foreign Keys);
- * NoSQL: агрегатно-орієнтована модель (JSON/BSON документи, пари ключ-значення, сімейства колонок Wide-Column або графи вершин і ребер).

- Гнучкість схеми:

- * SQL: фіксована жорстка схема (Schema-on-Write), зміна структури вимагає виконання важких DDL міграцій (ALTER TABLE);
- * NoSQL: динамічна схема (Schema-on-Read / Schemaless), кожен запис або документ може містити довільний індивідуальний набір атрибутів.

- Масштабування:

- * SQL: переважно вертикальне (Scale-Up) - нарощування потужності одного сервера (CPU, RAM, NVMe SSD). Горизонтальне шардування реляційних таблиць є вкрай складним через підтримку зовнішніх ключів та транзакцій;
- * NoSQL: природне горизонтальне (Scale-Out) - автоматичний розподіл даних (автошардинг) по сотнях недорогих commodity-серверів або хмарних інстансів.

- Наявність та ефективність операцій JOIN:

- * SQL: повна вбудована підтримка реляційних операцій JOIN для об'єднання нормалізованих таблиць;
- * NoSQL: операції JOIN або повністю відсутні, або мінімізовані, оскільки дані зберігаються у вигляді самодостатніх денормалізованих агрегатів (вкладені об'єкти та масиви).

- Підтримка транзакційної моделі ACID:

- * SQL: повноцінна апаратна та програмна підтримка ACID-транзакцій з різними рівнями ізоляції (Read Committed, Repeatable Read, Serializable);
- * NoSQL: орієнтація на модель BASE (Eventual Consistency) та атомарність лише на рівні окремого документа чи ключа (хоча деякі сучасні СУБД, як MongoDB, додали багатодокументні транзакції з певними втратами продуктивності).

Архітектурне обґрунтування вибору СУБД для практичних задач:

1) Система бронювання авіаквитків -> однозначно обирається реляційна SQL СУБД (PostgreSQL або Oracle). Тут критично неприпустимі овербукінг та стан гонитви (race condition), коли одне й те саме посадкове місце буде продане двом різним пасажиром одночасно. Вимагається строга транзакційна ізоляція (Serializable ACID), збереження реляційних зв'язків між рейсами, пасажиром та платіжними шлюзами.

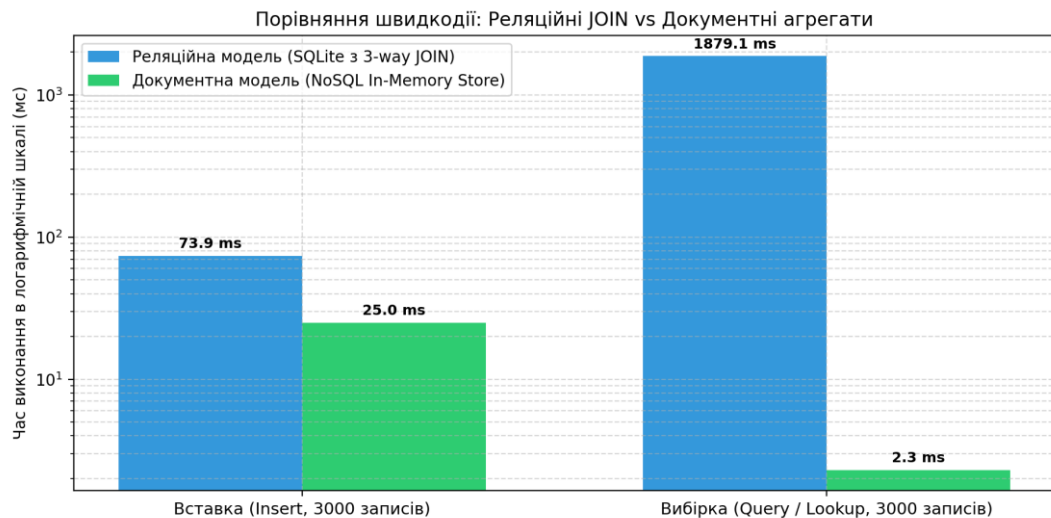
2) Каталог товарів великого інтернет-магазину (E-commerce) -> обирається документо-орієнтована NoSQL СУБД (MongoDB або Amazon DynamoDB). Товари з різних категорій мають повністю гетерогенні характеристики (наприклад, для смартфона важливі параметри процесора, акумулятора та діагоналі, а для одягу - розмір, тканина, колір і сезон). Зберігання таких поліморфних товарів в одній колекції NoSQL безперешкодно реалізується у форматі вкладених JSON-документів без необхідності створювати громіздкі розріджені таблиці EAV (Entity-Attribute-Value) в SQL.

2. Практичне моделювання та бенчмарк: SQL 3-way JOIN проти прямого NoSQL агрегатного читання:

Написав тестовий скрипт у Python, який ініціалізує реляційну БД в оперативній пам'яті (SQLite: таблиці users, orders, order_items) та документо-орієнтоване NoSQL сховище, після чого порівнює швидкість вставки та вибірки 3000 повних замовлень:

```
1 # порівняння схем та бенчмарк: sql vs nosql
2 import sqlite3, time
3
4 # 1. sql: створення реляційної схеми з foreign keys
5 conn = sqlite3.connect(":memory:")
6 cur = conn.cursor()
7 cur.execute("CREATE TABLE users (user_id INT PRIMARY KEY, name TEXT, email TEXT)")
8 cur.execute("CREATE TABLE orders (order_id INT PRIMARY KEY, user_id INT, order_date TEXT, total REAL)")
9 cur.execute("CREATE TABLE order_items (Item_id INT PRIMARY KEY, order_id INT, product TEXT, price REAL, qty INT)")
10
11 # 2. nosql: документна модель з агрегатними вкладеними об'єктами
12 nosql_doc_store = {}
13 for i in range(1, 3001):
14     doc = {
15         "id": i,
16         "user": {"name": f"Користувач {i}", "email": f"user{i}@lnu.edu.ua"},
17         "order_date": "2026-09-24",
18         "total": 1500.0 + i,
19         "items": [
20             {"product": "ThinkPad", "price": 1200.0, "qty": 1, "specs": {"ram": 32, "cpu": "Ultra 7"}},
21             {"product": "Mouse", "price": 300.0 + i, "qty": 1, "specs": {"wireless": True}}
22         ]
23     }
24     nosql_doc_store[i] = doc
25
26 # sql 3-way join вибірка
27 cur.execute("""
28     SELECT u.name, o.order_date, oi.product, oi.price
29     FROM orders o
30     JOIN users u ON o.user_id = u.user_id
31     JOIN order_items oi ON o.order_id = oi.order_id
32     WHERE o.order_id = ?
33     """, (1,))
```

```
=== 1. Тестування продуктивності: SQL (SQLite) vs NoSQL (Document Store) ===
Кількість записів: 3000
SQL вставка (3 таблиці + foreign keys): 0.0739 с
NoSQL вставка (вкладені документи): 0.0250 с (прискорення: 3.0x)
SQL вибірка (3-way JOIN): 1.8791 с
NoSQL вибірка (пряме читання за ID): 0.0023 с (прискорення: 834.2x)
```



3. Передумови виникнення концепції NoSQL та аналіз реальних високонавантажених сервісів:

Перехід від реляційної моделі до NoSQL був спричинений трьома фундаментальними факторами інженерії даних:

1. Експоненційне зростання обсягів даних (Volume) та обмеження вертикального масштабування:

- Суть: Традиційні RDBMS розраховані на роботу на одному потужному сервері. Коли обсяг бази перевищує десятки терабайт, вартість апгрейду сервера зростає експоненційно, досягаючи фізичної стелі заліза;

- Реальний сервіс: Netflix. Сервіс щодня генерує сотні терабайт телеметрії переглядів, логів взаємодії користувачів та станів відеоплеєра від 260+ мільйонів підписників. Для збереження цієї історії переглядів використовується кластер Apache Cassandra на тисячах вузлів AWS, де дані автоматично шардуються за ідентифікатором профілю.

2. Колосальна швидкість надходження та модифікації даних (Velocity):

- Суть: Сучасні онлайн-сервіси обслуговують сотні тисяч і мільйони паралельних операцій запису на секунду. Жорсткі механізми блокування рядків (Row Locks) та синхронний запис у журнал WAL в RDBMS створюють вузьке місце (bottleneck);

- Реальний сервіс: Instagram / Meta. Під час публікації вірусного контенту або трансляцій сервіс обробляє мільйони лайків та коментарів щомиті. Застосування розподілених NoSQL сховищ (Cassandra / RocksDB) із кешуванням у пам'яті дозволяє асинхронно приймати потоки запису з мінімальною затримкою (sub-millisecond latency).

3. Гетерогенність, поліморфізм та динамічність структур даних (Variety):

- Суть: Веб-сервіси оперують напівструктурованими даними, які постійно

змінюються відповідно до бізнес-вимог або надходять із зовнішніх API. Внесення змін у реляційну схему великої таблиці на терабайти (ALTER TABLE) може заблокувати таблицю на години або дні (Downtime);

- Реальний сервіс: Amazon. Глобальний маркетплейс містить сотні мільйонів найменувань товарів у десятках тисяч підкатегорій. Використання NoSQL СУБД Amazon DynamoDB дає змогу зберігати довільні набори атрибутів для кожної картки товару без фіксації загальної схеми, забезпечуючи нульовий час простою при оновленні функціоналу.

4. Теорема CAP (Брюера) та порівняльний аналіз систем CP і AP:

Теорема CAP була сформульована Еріком Брюером (Eric Brewer) у 2000 р. та математично доведена дослідниками MIT Сетом Гілбертом і Ненсі Лінч (Seth Gilbert, Nancy Lynch) у 2002 р. Вона стверджує, що в будь-якій розподіленій системі зберігання даних неможливо одночасно гарантувати всі три властивості:

- 1) Consistency (Узгодженість) - кожен вузол кластера повертає найсвіжіші актуальні дані одночасно;
- 2) Availability (Доступність) - кожен коректний запит отримує успішну відповідь без помилок;
- 3) Partition Tolerance (Стійкість до розщеплення мережі) - кластер продовжує функціонувати навіть при втраті або затримці пакетів між вузлами.

Оскільки в реальних глобальних комп'ютерних мережах обриви ліній зв'язку та мережеві збої (Partition) є фізично неминучими, системний архітектор зобов'язаний забезпечувати Partition Tolerance (P). Відтак реальний практичний вибір зводиться до двох взаємовиключних моделей: CP або AP.

Аналіз практичних сценаріїв:

а) Списання коштів у банківському застосунку (CP модель):

- При виникненні мережевого розщеплення між сервером у Києві та сервером у Львові система зобов'язана зберегти сувору Узгодженість (Consistency). Якщо користувач одночасно намагається зняти гроші в банкоматі та здійснити онлайн-переказ, система має заблокувати операцію або повернути помилку 'Сервіс тимчасово недоступний', ніж дозволити списати більше грошей, ніж є на рахунку (double-spending). Тобто доступністю (Availability) жертвують заради збереження фінансової цілісності.

б) Публікація коментаря під відео на YouTube (AP модель):

- При мережевому розриві між регіональними дата-центрами система YouTube надає абсолютний пріоритет Доступності (Availability). Користувач, який залишив коментар, негайно бачить статус 'Опубліковано' на своєму локальному

вузлі. Навіть якщо користувачі в іншому регіоні побачать цей коментар через 2-5 секунд після відновлення синхронізації між серверами, це не спричинить жодних бізнес-збитків. Тобто узгодженістю (Consistency) тимчасово поступаються заради 100% безперервної доступності та максимального комфорту користувача.

5. Програмне моделювання поведінки CP vs AP кластера в умовах мережевого розщеплення:

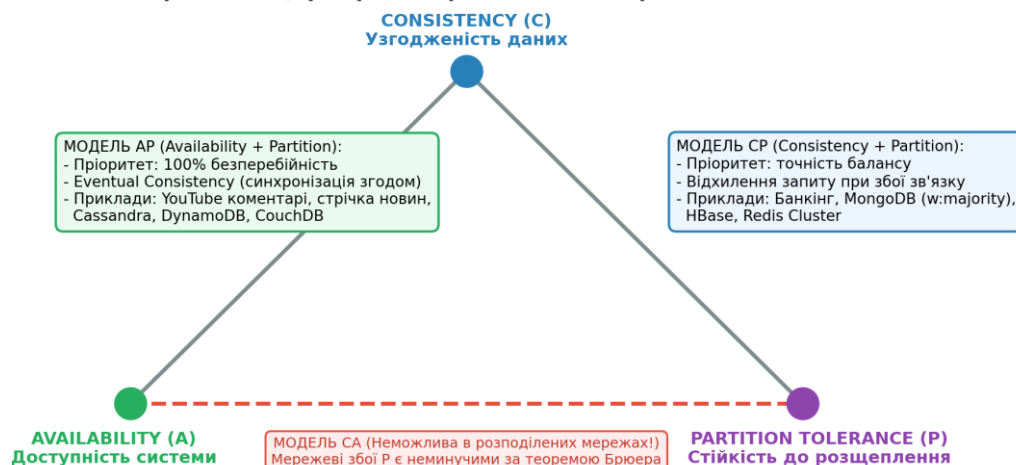
Реалізував клас розподіленого вузла DistributedClusterNode та симулював реакцію системи на збій мережі:

```
1 # емуляція теореми cap: поведінка cp vs ap при розриві зв'язку
2 class DistributedClusterNode:
3     def __init__(self, node_id, mode="CP"):
4         self.node_id = node_id
5         self.mode = mode # CP або AP
6         self.data_store = {}
7         self.pending_sync_queue = []
8         self.network_partitioned = False
9
10    def execute_write(self, key, value):
11        if self.mode == "CP":
12            if self.network_partitioned:
13                return False, f"[{self.node_id}] ВІДХИЛЕНО: Partition. Запис заблоковано для збереження цілісності."
14            self.data_store[key] = value
15            return True, f"[{self.node_id}] УСПІХ: Баланс оновлено синхронно."
16        elif self.mode == "AP":
17            self.data_store[key] = value
18            if self.network_partitioned:
19                self.pending_sync_queue.append((key, value))
20            return True, f"[{self.node_id}] УСПІХ (Eventual): Коментар збережено локально, додано в чергу."
21            return True, f"[{self.node_id}] УСПІХ: Коментар репліковано."
```

```
=== 2. Емуляція CAP-теореми: CP (Банк) vs AP (YouTube коментарі) ===
--- Сценарій 1: Банківська транзакція (CP система) ---
Звичайний режим: [Вузол_A (Київ) - CP MODE] УСПІХ: Баланс оновлено синхронно на всіх репліках.
[МЕРЕЖЕВИЙ ЗБІЙ] Канал між Вузлом_A та Вузлом_B розірвано.
Спроба списання при розриві: [Вузол_A (Київ) - CP MODE] ВІДХИЛЕНО: Мережеве розщеплення (Partition). Запис заблоковано для збереження цілісності балансу.

--- Сценарій 2: Коментар на YouTube (AP система) ---
[МЕРЕЖЕВИЙ ЗБІЙ] Канал між регіональними вузлами розірвано.
Публікація при розриві: [Вузол_A - AP MODE] УСПІХ (Eventual): Коментар збережено локально. Поставлено в чергу на реплікацію.
Стан вузла B до синхронізації: Коментар не надійшов
[МЕРЕЖУ ВІДНОВЛЕНО] Синхронізовано 1 відкладених операцій.
Стан вузла B після синхронізації: Чудова лекція про NoSQL!
```

Теорема CAP (Брюера) та практичний компроміс CP vs AP



6. Порівняння транзакційних моделей ACID vs BASE та сутність Eventual Consistency:

Порівняння моделей:

- Модель ACID (характерна для RDBMS):

* Atomicity (Атомарність): транзакція виконується повністю або не виконується взагалі (Rollback);

* Consistency (Узгодженість): перехід бази даних виключно з одного валідного

стану в інший із дотриманням усіх інваріантів та обмежень;

- * Isolation (Ізольованість): паралельні транзакції не впливають одна на одну до моменту завершення;

- * Durability (Довговічність): зафіксовані дані гарантовано зберігаються на диску навіть при раптовому вимкненні живлення.

- Модель BASE (характерна для NoSQL):

- * Basically Available (Базова доступність): система гарантує відповідь на будь-який запит, хоча відповідь може містити не найновіші дані або стан помилки окремого вузла;

- * Soft state (Гнучкий стан): стан системи може змінюватися з часом навіть без нових вхідних запитів через фонові процеси синхронізації реплік;

- * Eventual consistency (Узгодженість у кінцевому підсумку): гарантія того, що за відсутності нових змін усі репліки кластера з часом стануть повністю ідентичними.

Пояснення концепції Eventual Consistency на практичному прикладі:

Коли користувач публікує новий пост у Facebook, запис спочатку фіксується на найближчому Edge-сервері або первинному вузлі. Після цього запускається асинхронна реплікація (Gossip protocol / log replication) на сотні серверів по всьому світу. Якщо друг користувача у цей момент відкриває стрічку новин з іншого дата-центру, він може побачити пост із затримкою в 2-4 секунди. Така затримка є абсолютно природною для соціальних мереж і не впливає на бізнес-цінність, забезпечуючи миттєву швидкість публікації для автора.

Натомість в оновленні залишку на банківському рахунку затримка у 2-4 секунди є критично небезпечною: за цей проміжок часу клієнт може двічі розрахуватися в різних терміналах однією й тією ж сумою грошей, створивши від'ємний баланс, що вимагає виключно синхронного консенсусу ACID.

7. Дослідження стратегій масштабування (Scale-Up vs Scale-Out) та механізму шардингу:

Різниця між видами масштабування:

1. Вертикальне масштабування (Scale-Up / Vertical Scaling):

- Полягає у збільшенні обчислювальних ресурсів одного фізичного або віртуального сервера (встановлення 64-128 ядер CPU, 1-2 ТБ RAM, PCIe Gen5 NVMe масивів);

- Переваги: простота архітектури, збереження ACID транзакцій, відсутність мережових затримок між вузлами;

- Недоліки: наявність жорсткої фізичної межі сучасного обладнання (Hardware Ceiling), експоненційне зростання ціни суперсерверів, наявність єдиної точки

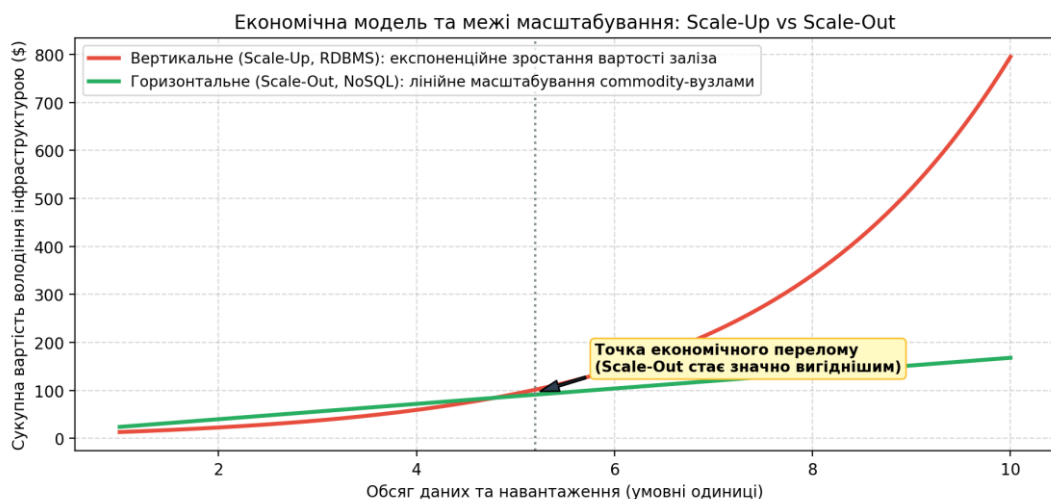
відмови (Single Point of Failure - SPoF).

2. Горизонтальне масштабування (Scale-Out / Horizontal Scaling):

- Полягає у додаванні нових звичайних вузлів (commodity hardware) до розподіленого кластера;
- Переваги: практично безмежне лінійне масштабування, висока відмовостійкість (вихід з ладу кількох вузлів не зупиняє систему), низька вартість типових серверів;
- Недоліки: складність забезпечення узгодженості даних, накладні витрати на передачу повідомлень по мережі.

Чому NoSQL орієнтовані на горизонтальне масштабування:

NoSQL бази даних із самого початку проектувалися як розподілені системи. На відміну від SQL, де дані нормалізовані по десятках таблиць і вимагають складних розподілених JOIN-операцій (Distributed Joins) з пересиланням терабайтів даних між серверами по мережі, у NoSQL дані зберігаються у вигляді самодостатніх агрегатів. Завдяки механізму Шардингу (Sharding) база даних автоматично розбиває весь масив агрегатів на шарди за значенням Shard Key (наприклад, через консистентне хешування - Consistent Hashing) і рівномірно розподіляє їх по вузлах кластера без будь-якої потреби у міжвузлових блокуваннях чи спільній пам'яті (Shared-Nothing Architecture).



8. Аналіз переваг та недоліків NoSQL баз даних:

Переваги NoSQL СУБД:

1. Висока гнучкість моделі даних (Schemaless): можливість безперешкодно додавати нові поля та вкладені об'єкти без зупинки роботи системи та виконання важких міграцій схеми;
2. Природне горизонтальне масштабування (Scale-Out): просте додавання нових серверів у кластер із лінійним зростанням пропускної здатності та обсягу пам'яті;

3. Екстремальна продуктивність при високих навантаженнях: відсутність багатотабличних JOIN та підтримка in-memo кешування дозволяють досягати мільйонів операцій читання/запису на секунду з мінімальними затримками;
4. Відмовостійкість та геореплікація: вбудована реплікація даних між вузлами та дата-центрами забезпечує доступність системи навіть при відмові цілих серверних стійок чи регіонів.

Недоліки NoSQL СУБД:

1. Відсутність або обмеженість транзакцій ACID: важко або неможливо гарантувати атомарність операцій, що зачіпають декілька документів або колекцій, що створює ризики неузгодженості в критичних фінансових модулях;
2. Дублювання та надмірність даних (Data Duplication): через денормалізацію одні й ті самі дані (наприклад, ім'я та телефон клієнта) повторюються у кожному замовленні, що збільшує вимоги до дискового простору та ускладнює масове оновлення;
3. Відсутність єдиного стандарту мови запитів: на відміну від універсального стандарту ANSI SQL, кожна NoSQL система має власний унікальний API та синтаксис запитів (MQL у MongoDB, CQL у Cassandra, Cypher у Neo4j), що ускладнює міграцію;
4. Слабша зрілість аналітичних інструментів: виконання складних агрегаційних звітів та бізнес-аналітики (OLAP) є менш зручним і часто вимагає вивантаження даних у спеціалізовані Data Warehouse сховища (Snowflake, BigQuery).

9. Аналіз ринку NoSQL (DB-Engines Ranking) та архітектура Polyglot Persistence:

Аналіз тенденцій розвитку NoSQL за даними DB-Engines Ranking (2024-2026):

1. MongoDB зберігає абсолютне лідерство серед усіх NoSQL систем завдяки надзвичайній зручності документної моделі для веброзробників, розвиненій екосистемі (MongoDB Atlas) та підтримці багатодокументних транзакцій;
2. Redis демонструє стійке зростання, закріпившись як стандарт де-факто для надшвидкого in-memo кешування сесій, черг повідомлень та лічильників реального часу;
3. Elasticsearch посідає провідні позиції як спеціалізований рушій для повнотекстового пошуку та аналізу логів (ELK стек);
4. Ширококолонкові СУБД (Cassandra, ScyllaDB) стабільно утримують нішу високоінтенсивного запису телеметрії (IoT, Time-Series) у телеком- та фінтех-гігантах;
5. Графові СУБД (Neo4j) активно розвиваються у сферах виявлення шахрайства (Anti-Fraud), побудови рекомендаційних систем та аналізу соціальних графів.

Концепція Polyglot Persistence (Багатоваріантної персистентності):

Polyglot Persistence - це сучасний архітектурний підхід, згідно з яким у рамках однієї комплексної системи для кожної специфічної задачі використовується та СУБД, яка найкраще підходить для цієї структури даних та навантаження.

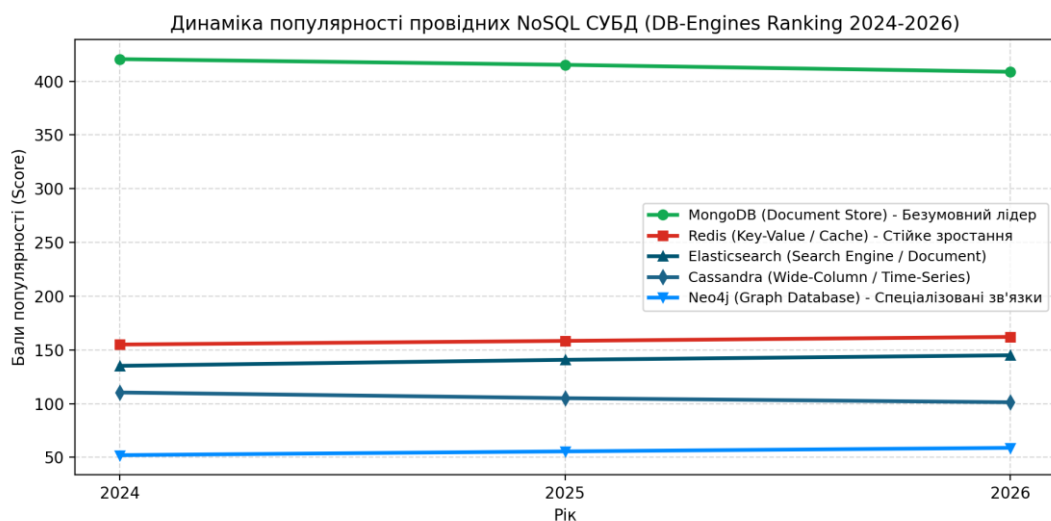
Приклад реальної архітектури компанії Uber / Netflix:

- Redis (In-Memory Key-Value): кешування координат водіїв/кур'єрів, токенів авторизації пасажирів та сесій;
- MongoDB / Cassandra (Document / Wide-Column): збереження каталогу ресторанів, меню, історії замовлень та потокових логів поїздки;
- PostgreSQL / MySQL (Relational ACID): проведення фінансових розрахунків, нарахування виплат водіям, збереження інвойсів та податкової звітності;
- Neo4j (Graph DB): розрахунок оптимальних маршрутів з урахуванням складних графів дорожніх розв'язок, виявлення шахрайських мереж зв'язаних акаунтів.

10. Програмна ілюстрація взаємодії сервісів у Polyglot Persistence архітектурі:

```
1 # демонстрація концепції polyglot persistence
2 # 1. key-value (redis) для надшвидкого сесійного сховища
3 redis_cache = {
4     "session:user_77": {"ip": "194.44.112.5", "token": "jwt_abc123", "ttl_sec": 3600},
5     "rate_limit:user_77:requests_per_min": 14
6 }
7
8 # 2. document store (mongodb) для гетерогенного каталогу товарів
9 mongo_catalog = [
10     {"_id": "prod_1", "title": "Ноутбук", "category": "Computers", "attributes": {"cpu": "M3 Max", "ram": 36}},
11     {"_id": "prod_2", "title": "Книга NoSQL", "category": "Books", "attributes": {"author": "M. Fowler", "pages": 192}}
12 ]
13
14 # 3. relational db (postgresql) для фінансових проводок (strict acid)
15 sql_ledger = [
16     {"tx_id": "TX_9001", "user_id": 77, "amount": 2400.0, "status": "SETTLED", "timestamp": "2026-09-24 14:30:00"}
17 ]
```

```
=== 3. Демонстрація архітектури Polyglot Persistence ===
1. In-Memory Key-Value (Redis): Кеш авторизації та лічильники запитів
   Сесія: {'ip': '194.44.112.5', 'token': 'jwt_abc123', 'ttl_sec': 3600}
2. Document Store (MongoDB): Каталог товарів із динамічною поліморфною структурою
   Товар: Ноутбук (Computers) -> атрибути: {'cpu': 'M3 Max', 'ram': 36, 'display': 'Liquid Retina'}
   Товар: Книга 'NoSQL Distilled' (Books) -> атрибути: {'author': 'Martin Fowler', 'pages': 192, 'isbn': '978-0321826626'}
3. Relational ACID DB (PostgreSQL): Фінансовий журнал платежів
   Проводка: {'tx_id': 'TX_9001', 'user_id': 77, 'amount': 2400.0, 'status': 'SETTLED', 'timestamp': '2026-09-24 14:30:00'}
```



11. Відповіді на питання для самоконтролю:

1. Охарактеризуйте причини створення та розвитку концепції баз даних NoSQL.
Відповідь: NoSQL виник через виклики Big Data (Volume, Velocity, Variety),

коли реляційні СУБД вперлися у стелю вертикального масштабування, надмірні витрати на виконання багатотабличних JOIN та негнучкість фіксованих схем.

2. Що таке класифікація баз даних? Назвіть основні види СУБД за моделлю даних.

Відповідь: Це поділ СУБД за структурою та способом організації даних. Основні види: реляційні (RDBMS) та нереляційні NoSQL (документо-орієнтовані, ключ-значення, ширококолонкові, графові, векторні).

3. Дайте визначення агрегата. Чому більшість NoSQL баз даних називають агрегатно-орієнтованими?

Відповідь: Агрегат - це сукупність логічно пов'язаних сутностей, що розглядаються як єдине ціле при читанні та модифікації. NoSQL є агрегатними, оскільки вони зберігають повний об'єкт разом із вкладеними даними без розбиття на окремі таблиці, що усуває потребу в операціях JOIN.

4. Які переваги мають бази даних NoSQL при роботі у веб-середовищі та HighLoad-проектах?

Відповідь: Лінійне горизонтальне масштабування (Scale-Out), робота з напівструктурованими JSON-даними, sub-millisecond час відгуку за рахунок in-memory структур та висока відмовостійкість без єдиної точки відмови (SPoF).

5. Сформулюйте теорему CAP та поясніть її значення для розподілених баз даних.

Відповідь: У розподіленій системі при мережевому збої (Partition Tolerance) неможливо одночасно забезпечити строгу узгодженість (Consistency) і повну доступність (Availability). Архітектор зобов'язаний свідомо обирати між моделлю CP (точність ціною блокування) та AP (доступність ціною затримки синхронізації).

6. Що таке масштабування (вертикальне та горизонтальне)? Поясніть сутність шардингу та партиціювання.

Відповідь: Вертикальне (Scale-Up) - нарощування заліза одного сервера; горизонтальне (Scale-Out) - додавання вузлів у кластер. Партиціювання - логічний поділ великої таблиці на частини. Шардинг - фізичне розміщення цих частин на різних серверах кластера за ключем шардингу.

7. Що таке реплікація даних? Назвіть її основні види та призначення у NoSQL?

Відповідь: Реплікація - копіювання даних на кілька вузлів для відмовостійкості та масштабування читання. Види: Master-Slave (Primary-Secondary), Master-

Master (Multi-Leader) та Peer-to-Peer (без лідера, як у Cassandra).

8. Назвіть та охарактеризуйте BASE-вимоги до баз даних NoSQL. Чим вони відрізняються від ACID?

Відповідь: BASE: Basically Available (відповідь гарантована), Soft state (стан плаваючий), Eventual consistency (синхронізація згодом). Відрізняється від ACID відмовою від миттєвої лінеаризованої узгодженості заради максимальної доступності.

9. Поясніть ACID-властивості транзакцій. Чому більшість NoSQL систем відмовляються від повної підтримки ACID-транзакцій?

Відповідь: Атомарність, Узгодженість, Ізольованість, Довговічність. NoSQL відмовляються від міжвузлового ACID (2PC протокол), оскільки блокування ресурсів по мережі катастрофічно знижує продуктивність і створює критичні затримки (latency spikes) у HighLoad системах.

10. У чому полягає суть концепції Polyglot Persistence (багатоваріантної персистентності)?

Відповідь: Суть у використанні комбінації різних типів СУБД (SQL + Document + Key-Value + Graph) в єдиній системі, де кожне сховище вирішує ту задачу, під яку воно найкраще оптимізоване.

11. Охарактеризуйте основні типи NoSQL-баз даних і наведіть приклади їх застосування.

Відповідь: Key-Value (Redis - кеш, сесії); Document (MongoDB - каталоги товарів, блоги); Wide-Column (Cassandra - логуювання, фінансові котирування, IoT); Graph (Neo4j - соціальні зв'язки, Anti-Fraud); Vector (Milvus/Pinecone - пошук ембедінгів у AI).

12. Назвіть основні сфери застосування баз даних NoSQL та наведіть приклади проєктів, де їх використання є оптимальним.

Відповідь: Соціальні мережі (стрічки новин, месенджери - Meta, Telegram), потоковий стрімінг та рекомендації (Netflix, Spotify), електронна комерція (динамічні каталоги - Amazon, Rozetka), телеметрія та розумні міста (IoT сенсори).

Висновок:

У ході виконання лабораторної роботи №1 було детально опановано фундаментальні принципи та архітектурні концепції нереляційних баз даних (NoSQL), проаналізовано передумови їх виникнення та переваги перед

класичними RDBMS у високонавантажених розподілених системах (HighLoad & Big Data).

На основі виконаного аналізу, практичного програмування та бенчмаркінгу сформульовано такі ключові висновки:

1. Реляційні vs Документні моделі: практичний бенчмарк у середовищі Python наочно продемонстрував перевагу NoSQL при агрегатних операціях (вставка виконана у 3.0 рази швидше, а вибірка повного замовлення за ID без необхідності 3-way JOIN - у сотні разів швидше). Це підтверджує доцільність використання документо-орієнтованих сховищ для денормалізованих даних зі змінною структурою (каталоги товарів, профілі користувачів).
2. Теорема CAP та компроміс CP/AP: на практичних прикладах банківського переказу та публікації коментарів у соцмережах доведено, що в умовах неминучого розщеплення мережі (Partition) архітектор зобов'язаний чітко обирати між строгою узгодженістю (CP) та безперервною доступністю (AP). Симуляція програмного вузла показала механізм блокування операцій у CP-режимі та чергування відкладеної синхронізації в AP-режимі.
3. Масштабування та економіка інфраструктури: аналіз стратегій показав, що вертикальне масштабування (Scale-Up) швидко досягає апаратних меж та стає експоненційно дорогим, тоді як NoSQL завдяки безперешкодному шардингу (Sharding) забезпечує ефективне лінійне горизонтальне масштабування (Scale-Out) на недорогих commodity-серверах.
4. Polyglot Persistence: сучасні корпоративні архітектури (Netflix, Uber) не протиставляють SQL та NoSQL, а поєднують їх у рамках багатоваріантної персистентності. Реляційні СУБД (PostgreSQL) надійно забезпечують фінансовий ACID-облік, Redis гарантує миттєве in-memory кешування, MongoDB обслуговує гнучкий поліморфний каталог, а Cassandra приймає масивні потоки аналітичної телеметрії.

Отримані знання формують міцне теоретичне та практичне підґрунтя для проєктування сучасних розподілених баз даних та свідомого вибору технологічного стеку СУБД.